

Personal AI-Assisted Engineering Workflow

Workplace Edition - Global Personal Operating System

A reusable, risk-based workflow for using agentic AI as a force multiplier while keeping engineering judgment, correctness, security, architecture, and trade-offs under human ownership.

Important: this is a personal workflow, not a team process. It is designed to be installed once and used across every repository you work on. Teammates may use completely different processes.

```
Human intent
-> change classification
-> security impact check
-> documentation impact check
-> human-owned requirements
-> AI challenge
-> human decision
-> explicit GO
-> AI implementation
-> machine verification
-> adversarial tracks as applicable
-> independent review
-> documentation reconciliation
-> human final approval
```

Core rule

AI may reduce typing. It must not reduce the amount of engineering reasoning the human performs.

1. The Architecture in One View

The workflow is split into three layers so that your personal process does not become part of every application repository.

Layer	Where it lives	Purpose
Personal workflow	Workflow Git repository + personal Claude install	Portable install, global contract, model profiles, installer, templates
Target repository	Normal project repository	Team code, team CLAUDE.md, existing ADRs/docs/tests/code
Private project state	<code>\$ENGINEERING_WORKFLOW_HOME/projects/<i>name</i>/private</code>	My project notes, private decisions, architecture notes, secrets

The repository is reality

Private workflow state is continuity and reasoning support. It never outranks current code, tests, runtime evidence, team-shared contracts, approved team ADRs, or production evidence. When your notes disagree with reality, the workflow reports the disagreement instead of silently changing either side.

Why this matters

- You can use the same engineering workflow on Go, Node.js, or a mixed-language service without adding workflow files to the project.
- A teammate can work differently without breaking your workflow.
- Team changes can invalidate your private assumptions, and the workflow is designed to detect that drift.

2. Human Authority and the GO Gate

The central boundary is stronger than "AI writes, human reviews." The human controls meaning; the agent controls execution of approved work.

Human owns	AI may assist with
Requirements and acceptance criteria	Gap detection, ambiguity detection, candidate requirements - clearly
Invariants and correctness semantics	Failure sequences, counterexamples, test ideas
Architecture and trade-offs	Alternatives, risk analysis, dependency analysis
Security acceptance and risk decisions	Threat-model attacks and concrete exploit paths
Final approval	Implementation, verification, evidence gathering, document reconcili

Write authorization protocol

```
PROPOSE
show intended changes
show affected files
show risks / side effects
show verification plan
|
v
HUMAN GO
|
v
WRITE
|
v
VERIFY
```

Skill invocation is not GO. Invoking an implementation or documentation skill tells the agent which role to use; it does not grant permission to write.

This same principle applies to documentation. The documentation guardian always shows the proposed patch/content first, then waits for GO before modifying private or shared documentation.

3. Risk-Based Gates - Full Model, Variable Depth

The workflow keeps the complete engineering model but scales ceremony with risk. Every meaningful gate is explicitly reported as REQUIRED, N/A WITH REASON, or OPTIONAL WITH REASON. Important concerns are never silently skipped.

Class	Typical meaning	Default route
T0	Purely mechanical: formatting, comments, type, no behavior, no side effects	no docs
T1	Localized low-risk behavior; no contract, persistence, privilege, rights, change, side effects, or major	no docs
T2	Meaningful behavior/API/state/persistence/contract/observability/performance/appliable specialist	no docs
T3	Security boundary, sensitive data, migration, full workflow coordination, decision gate effects, the	no docs

Security is different

Security is considered for every meaningful change. A security gate may be N/A only with concrete reasoning showing that the change cannot affect a security boundary, input validation, privilege, secrets, resource limits, sensitive data handling, trust assumptions, or external effects. A tiny change can therefore escalate if its downstream impact is meaningful.

4. The Normal Feature Workflow

For T2/T3 work, the canonical route is:

1. Human intent and scope
2. Change classification
3. Security impact check
4. Human-owned requirements
5. Requirements Architect
6. Human security pre-flight
7. System Design Challenger
8. Security Adversary
9. Distributed Tracing Engineer (when distributed/asynchronous)
10. Human decision / decision record where warranted
11. Explicit GO
12. Implementation Agent
13. Runtime Engineer (when relevant)
14. Verification Engineer
15. Distributed Adversary (when distributed)
16. Security implementation review (when applicable)
17. Trace validation (when applicable)
18. Senior Code Review
19. Engineering Reasoning Review
20. Documentation Guardian
21. Human final approval
22. Private workflow state update

What changes for T0/T1?

Only the depth changes. The orchestrator still evaluates security and downstream side effects first. For a truly trivial change it may conclude that architecture, distributed correctness, tracing, documentation, and other heavyweight stages are N/A with explicit reasons.

The workflow must not become ceremonial. Its purpose is to preserve engineering cognition and make evidence visible, not to maximize the number of Markdown files or AI calls.

5. The 16 Skills

Skill	Purpose
engineering-orchestrator	Routes the change, enforces gates, and makes risk classification explicit.
requirements-architect	Facilitates human-owned requirements and acceptance criteria.
system-design-challenger	Attacks architecture, ownership, failure semantics, scalability, and hidden assumptions.
implementation-agent	Implements approved scope only.
runtime-engineer	Explains language/runtime behavior; works for Go, Node.js, or other project runtimes.
verification-engineer	Builds evidence through proportional tests and checks.
distributed-adversary	Attacks delay, loss, duplication, reordering, stale messages, ownership races, and rec.
chaos-engineer	Runs controlled, isolated fault experiments.
performance-engineer	Measures first, forms a hypothesis, changes, and re-measures.
senior-code-reviewer	Independent review against requirements, decisions, correctness, security, operations
ai-incident-commander	Runs evidence-first production incident diagnosis.
security-adversary	Threat-models and attacks security assumptions before/after implementation.
distributed-tracing-engineer	Designs and validates causal telemetry across remote/async/retry/recovery boundaries
engineering-reasoning-reviewer	Checks whether you can defend the change, guarantees, failure modes, and evidence
engineering-guide	Navigation and private-state/resume layer.
documentation-guardian	Detects material documentation drift and proposes changes before writing them.

6. Documentation Guardian - The Drift Problem

Documentation is treated as a living model, but the workflow intentionally keeps it sparse. The guardian exists because code can change faster than your private understanding - especially when somebody else changes a shared repository.

What it monitors

- Private requirements, decisions, architecture notes, security/tracing notes, runbooks, and workflow state.
- Existing team/shared repository docs such as README, ADRs, API contracts, operational guides, and configuration documentation.
- Code, tests, configuration, migrations, feature flags, Git history, and branch-sync changes as evidence.

Material documentation filter

- Changes service/module contracts or public behavior.
- Changes architecture, topology, invariants, security controls, persistence/data semantics, operations, compatibility, or durable capacity assumptions.
- Makes an existing important document factually wrong.
- Loses a durable decision or assumption that you will need to reason correctly later.

Review-first behavior

```
CHANGE / BRANCH SYNC
|
v
DETECT DRIFT
|
v
SHOW EVIDENCE + PROPOSED DIFF
|
v
HUMAN GO
|
v
APPLY ONLY APPROVED DOC CHANGES
|
v
RE-READ + VERIFY + RECORD
```

A one-line refactor that has no durable downstream effect should end with No material documentation impact. A one-line refactor that changes a critical invariant elsewhere must still be flagged.

7. Branch Sync is a First-Class Drift Event

A pull/rebase/merge from staging, production, release, or another authoritative branch is not merely a Git operation. It is new engineering input that may invalidate your assumptions.

```
git sync
-> inspect newly introduced relevant changes
-> compare against private model
-> compare against existing shared docs
-> flag stale requirements / decisions / architecture / security / tracing notes
-> show proposed reconciliation
-> HUMAN GO
-> apply approved documentation changes
-> resume implementation
```

Recommended command

```
/engineering-guide sync-check
```

Then, when material drift is found:

```
/documentation-guardian
```

The guardian should use available Git evidence such as ORIG_HEAD, reflow, merge/rebase diffs, or an equivalent pre/post-sync comparison. It must not assume the team preserved your private model.

Why the private/shared distinction matters

Your private decision record can say what *you* decided for your change. A team ADR can say what the organization agreed to. Those are different authorities. The guardian should identify contradictions between them rather than silently merging them into one narrative.

8. Private State: The Workflow Memory Outside Git

Each target repository gets a private workflow area keyed to the repository identity, preferably its stable Git remote; a normalized repository path is the fallback.

```
$ENGINEERING_WORKFLOW_HOME/  
projects/  
<project-id>/  
PROJECT.md  
WORKFLOW_STATE.md  
WORK_LOG.md  
requirements/  
decisions/  
architecture/  
security/  
tracing/  
incidents/  
performance/
```

Core files

- **WORKFLOW_STATE.md**: compact gate matrix, current stage, blockers, next action.
- **WORK_LOG.md**: append-only meaningful work history with evidence and human decisions.
- **requirements/, decisions/, security/, tracing/, etc.**: create only when the material is worth preserving.

No `docs/` directory is required in the application repository.

If your organization already has committed docs/ADRs, they remain available as team-shared input. The personal workflow does not erase, override, or assume ownership of those files.

9. Model Profiles - Strategy vs Implementation

Model selection is separated from engineering role definitions. Each skill declares a portable model profile, and each host adapter maps that profile to a concrete model.

Profile	Default role	Claude Code mapping
strategic	Planning, requirements, architecture, security, distributed reasoning	reasoning, incident diagnosis, senior review
implementation	Coding, routine verification, runtime-focused implementation	spoof, controlled experiments

The portable skill files do not hard-code Claude model IDs. They store ``metadata.workflow_model_profile`` plus a visible ``## Model`` section. The Claude installer injects the native ``model:`` field from one central mapping file.

```
hosts/claude/models.yaml

strategic: opus
implementation: sonnet
```

When models change, change the mapping once and reinstall/update. You do not have to edit 16 skills. A future Codex or other host adapter can map the same profiles to its own model names.

Claude Code currently documents both the ``model`` skill frontmatter field and personal skills under ``~/.claude/skills/``. The user-level ``~/.claude/CLAUDE.md`` is the appropriate scope for personal workflow instructions.

10. Global Claude Code Installation

This workflow is distributed as its own Git repository. The installer installs it globally; it does not modify the structure of your application repositories.

```
git clone <this-repository>
cd ai-assisted-engineering-workflow
./install.sh
```

The installer performs

- Copies the personal skills into `~/.claude/skills/`.
- Injects the Claude-specific `model:` alias and explicit invocation controls.
- Installs the workflow contract under `\$ENGINEERING\_WORKFLOW\_HOME`.
- Adds a user-level import to `~/.claude/CLAUDE.md` while creating a backup of any existing file.
- Installs helper scripts for private project state.

Verify

```
./scripts/verify-install.sh
```

Remove

```
./scripts/uninstall.sh
# or, only when intentionally deleting private state:
./scripts/uninstall.sh --purge-state
```

Nothing in the installer requires a project-local `~/.claude/skills/` directory. Claude Code documents personal skills as loading across your projects.

11. Daily Usage Patterns

Start of day / resumed work

```
/engineering-guide resume
```

New meaningful feature

```
/engineering-orchestrator
```

The orchestrator should first show something like:

```
Change class: T2
Security: REQUIRED
Distributed correctness: REQUIRED
Tracing: REQUIRED
Performance: OPTIONAL WITH REASON
Chaos: OPTIONAL WITH REASON
Documentation: REQUIRED

Next: requirements-architect
Human decision required: confirm route
```

Incident mode

```
/ai-incident-commander
```

The incident workflow is diagnosis-first: symptoms -> scope -> hypotheses -> evidence -> narrow reproduction -> root cause -> violated invariant -> remediation decision -> implementation -> verification -> postmortem.

Security-sensitive change

```
/security-adversary
```

The security skill starts from the human threat model, then attacks trust boundaries, authorization, input validation, resource exhaustion, replay/stale-state behavior, telemetry leakage, and recovery paths.

Performance change

```
/performance-engineer
```

Baseline -> hypothesis -> change -> measure -> compare -> review. No optimization based only on intuition.

12. What Makes This Different From a Coding Assistant

The workflow deliberately separates engineering cognition into distinct roles so that the same agent context is not simultaneously asked to design, implement, defend, and approve its own solution.

Stage	What AI does	What human retains
Requirements	Find omissions and ambiguity	Meaning, scope, acceptance criteria
Design	Attack assumptions and alternatives	Architecture and trade-offs
Implementation	Translate approved contract into code	Authorization to write; intended behavior
Verification	Generate/run tests and interpret evidence	Judgment about adequacy and residual risk
Adversarial review	Try to break correctness/security/operation	Diagnosis and decision about response
Documentation	Detect drift and prepare exact patch	Whether to change the documented model
Reasoning review	Probe your understanding	Final engineering understanding and accountability

The desired end state is not "AI can build the system for me." It is: "AI can make me faster while forcing me to reason about guarantees, assumptions, failure, security, observability, and trade-offs."

Host portability

```
PORTABLE SKILL
|
+-- Claude Code adapter -> native skill frontmatter + Opus/Sonnet mapping
+-- Future Codex adapter -> host skill mechanism + host model mapping
+-- Future host -> equivalent discovery / invocation / permission / model mapping
```

13. Source and Implementation Notes

This workplace edition preserves the source system's core principles: human-owned requirements, explicit challenge stages, evidence-first verification, security as a correctness property, distributed reasoning where architecture requires it, and a deliberate separation between portable skill specification and host integration.

Current Claude Code documentation used for the global installation design

- Skills: <https://code.claude.com/docs/en/skills>
- Memory / user-level CLAUDE.md: <https://code.claude.com/docs/en/memory>
- Agent Skills standard: <https://agentskills.io/>

Current Claude Code documentation states that personal skills placed under `~/.claude/skills//SKILL.md` load across your projects, while project skills live under `~/.claude/skills/`. It also documents the native `model` frontmatter field for per-skill model selection.

Installation repository contents

```
README.md
WORKFLOW.md
GLOBAL_ENGINEERING_CONTRACT.md
HOST_INTEGRATION_GUIDE.md
MODEL_PROFILES.md
STATE_LAYOUT.md
SKILLS.md
INSTALL.md

skills/<16 skill directories>/SKILL.md
hosts/claude/models.yaml
hosts/claude/skill-policy.yaml
templates/<private-state templates>
scripts/install helpers
```

The application repository remains untouched by the installer. Your only shared artifact may still be the team repository's existing `CLAUDE.md`, while your personal global contract and skills remain user-scoped.

Final rule

Do not confuse AI-generated code that works with an engineering system you understand, can explain, and can defend.